

Kod Anlatım Kopya Kağıdı

Proje adı: Dünyayı Gezayisun AI Rehberuylan

Kısa tanım: Strapi CMS, Python otomasyon ve Streamlit arayüzünden oluşan yapay zeka destekli, çift dilli gezi rehberi.

1. Hocaya 30 Saniyelik Özet

Bu projede veriler önce automation/data/places.json dosyasından okunuyor. automation/main.py bu verileri şehir ve mekan olarak işliyor. Şehir ve mekan açıklamaları yapay zeka ile zenginleştiriliyor, deep-translator ile İngilizceye çevriliyor, mekanlara uygun görseller Pollinations/HuggingFace ile üretiliyor. Sonra bu görseller Strapi Media Library'ye yükleniyor ve metinlerle beraber Strapi API'ye kaydediliyor. Streamlit tarafı da Strapi API'den şehirleri ve mekanları çekip kullanıcıya TR/EN seçeneekli modern bir arayüzde gösteriyor.

2. Genel Veri Akışı

```
places.json
↓
automation/main.py
↓
content_enricher.py → Yapay zeka ile açıklama zenginleştirme
translator.py       → TR / EN çeviri
image_generator.py  → Turistik görsel üretme
strapi_api.py       → Strapi'ye şehir, mekan ve görsel kaydetme
↓
Strapi CMS + Media Library + Cloudinary
↓
frontend-streamlit/api.py
↓
frontend-streamlit/app.py
↓
Kullanıcı arayüzü
```

Hocaya söyle:

Projede Python tarafı veri üretme ve Strapi'ye basma görevini yapıyor. Strapi veritabanı ve medya yönetimini tutuyor. Streamlit ise sadece bu verileri API'den okuyup kullanıcıya gösteriyor.

3. Strapi Veri Modeli

City Koleksiyonu

Dosya: strapi-backend/src/api/city/content-types/city/schema.json

Alanlar:

- name: Şehir adı
- name_en: Şehrin İngilizce adı
- country: Ülke adı
- country_en: Ülkenin İngilizce adı
- short_info: Türkçe kısa bilgi
- short_info_en: İngilizce kısa bilgi
- places: Bu şehre bağlı mekanlar

İlişki:

City.places → oneToMany → Place.city

Hocaya söyle:

City tarafında places alanı bir şehrin birden fazla mekana sahip olmasını sağlıyor. Bu alan mappedBy: city ile Place koleksiyonundaki city alanına bağlanıyor.

Place Koleksiyonu

Dosya: strapi-backend/src/api/place/content-types/place/schema.json

Alanlar:

- name: Mekan adı
- name_en: Mekanın İngilizce adı
- description_tr: Türkçe açıklama
- description_en: İngilizce açıklama
- rating: Puan

- cover_image: Kapak görseli
- city: Mekanın bağlı olduğu şehir

İlişki:

Place.city → manyToOne → City.places

Hocaya söyle:

Place tarafındaki city alanı many-to-one ilişki. Yani birçok mekan tek bir şehre bağlanabiliyor. Bu yüzden kullanıcı şehir seçtiğinde sadece o şehrin mekanlarını listeleyebiliyorum.

4. automation/main.py Ne İş Yapar?

Bu dosya otomasyonun ana dosyasıdır. Projenin veri hazırlama pipeline'ını başlatır.

Önemli akış:

1. .env dosyasından STRAPI_URL ve STRAPI_API_TOKEN bilgilerini okur.
2. Token veya URL yoksa hata verip durur.
3. automation/data/places.json dosyasından şehir ve mekan verilerini okur.
4. Her şehir için:
 - Şehir bilgisini yapay zeka ile zenginleştirir.
 - Şehir adını, ülkeyi ve kısa bilgiyi İngilizceye çevirir.
 - get_or_create_city() ile Strapi'de şehir varsa bulur, yoksa oluşturur.
5. Her mekan için:
 - Mekan açıklamasını yapay zeka ile zenginleştirir.
 - Açıklamayı İngilizceye çevirir.
 - Mekana uygun görsel üretir.
 - Görseli Strapi Media Library'ye yükler.
 - Mekan kaydını şehir ilişkisi ve görsel ile Strapi'ye ekler.

Hocaya söyle:

main.py orkestrasyon dosyası gibi çalışıyor. Diğer modülleri sırayla çağırıyor: önce veri okuma, sonra yapay zeka zenginleştirme, çeviri, görsel üretimi, Strapi upload ve kayıt oluşturma.

Ezber kod akışı:

```
enriched_short_info = enrich_city_info(...)
short_info_en = translate_text(...)
city_ref = strapi.get_or_create_city(...)
enriched_desc_tr = enrich_place_description(...)
desc_en = translate_text(...)
local_image_path = generate_and_save_image(...)
image_id = strapi.upload_image(...)
strapi.create_place(...)
```

5. automation/translator.py Ne İş Yapar?

Bu dosyada translate_text() fonksiyonu var.

Görevi:

- deep_translator kütüphanesindeki GoogleTranslator sınıfını kullanır.
- Türkçeden İngilizceye veya İngilizceden Türkçeye çeviri yapar.
- Hata olursa sistemi durdurmaz, orijinal metni geri döndürür.

Hocaya söyle:

Çeviri işlemlerini ayrı dosyaya aldım. Böylece hem şehir hem mekan çevirilerinde aynı fonksiyonu tekrar kullanıyorum. Çeviri başarısız olursa proje tamamen çökmesin diye fallback olarak orijinal metni döndürüyorum.

Önemli mantık:

```
translated = GoogleTranslator(source=source, target=target).translate(text)
```

6. automation/content_enricher.py Ne İş Yapar?

Bu dosya metinleri yapay zeka ile daha turistik ve açıklayıcı hale getirir.

Önemli fonksiyonlar:

- `_system_prompt()`: Yapay zekaya nasıl yazması gerektiğini söyler.
- `_clean_generated_text()`: Yapay zekadan gelen metindeki gereksiz markdown, madde işareti veya seçenek ifadelerini temizler.
- `_call_groq_text()`: Öncelikle Groq API ile metin üretmeye çalışır.
- `_call_pollinations_text()`: Groq çalışmazsa Pollinations ile metin üretir.
- `_generate_text()`: Birincil ve yedek yapay zeka stratejisini yönetir.
- `enrich_city_info()`: Şehir kısa bilgisini zenginleştirir.
- `enrich_place_description()`: Mekan açıklamasını zenginleştirir.

Hocaya söyle:

Burada amaç hazır gelen kısa açıklamaları daha doğal, turistik ve bilgilendirici hale getirmek. Öncelik Groq API'de. Groq başarısız olursa Pollinations deniyor. İkisi de başarısız olursa ana açıklama korunuyor.

Neden temizleme fonksiyonu var?

Yapay zeka bazen "Tabii, işte açıklama:" gibi gereksiz girişler veya markdown döndürebiliyor. `_clean_generated_text()` bunları temizleyip tek paragraf düzgün metin elde ediyor.

7. automation/image_generator.py Ne İş Yapar?

Bu dosya mekanlar için turistik/manzara tarzında görsel üretir.

Önemli fonksiyonlar:

- `slugify()`: Şehir ve mekan adını dosya adına uygun hale getirir.
- `_build_prompt()`: Görsel üretimi için İngilizce prompt hazırlar.
- `_generate_with_pollinations()`: Pollinations AI ile görsel üretir.
- `_generate_with_huggingface()`: Pollinations olmazsa HuggingFace ile dener.
- `_generate_with_picsum()`: AI servisleri olmazsa yedek görsel indirir.
- `generate_and_save_image()`: Tüm görsel üretim akışını yönetir.

Hocaya söyle:

Görsel üretirken önce mekan ve şehir adı İngilizceye çevriliyor. Çünkü görsel üretim modelleri İngilizce prompt ile daha iyi sonuç veriyor. Önce Pollinations deniyor, olmazsa HuggingFace, o da olmazsa picsum.photos fallback olarak kullanılıyor.

Önemli nokta:

Görsel önce yerelde automation/images içine kaydediliyor ama kalıcı olarak Strapi Media Library'ye yükleniyor. Yani değerlendirme açısından görsel Strapi'de duruyor, sadece lokal dosya upload için ara adım.

Ezber cümle:

Dosya yönetimi kısmında görseli sadece bilgisayarda bırakmadım; strapi_api.py içindeki `upload_image()` ile `/api/upload` endpoint'ine multipart olarak gönderdim.

8. automation/strapi_api.py Ne İş Yapar?

Bu dosya Python ile Strapi REST API arasındaki bağlantıyı yönetir.

Sınıf:

```
class StrapiAPI:
```

Ana görevleri:

- Token ile güvenli API isteği yapmak.
- Şehir var mı kontrol etmek, yoksa oluşturmak.
- Görseli Strapi Media Library'ye yüklemek.
- Mekan kaydını şehir ilişkisi ve kapak görseliyle oluşturmak.

Token Kullanımı

```
self.headers = {  
    "Authorization": f"Bearer {self.token}"  
}
```

Hocaya söyle:

Strapi API'ye istek atarken JWT/API Token kullanıyorum. Token kodun içine yazılmadı, .env veya Render environment variable üzerinden okunuyor.

get_or_create_city()

Ne yapar?

- Önce /api/cities endpoint'inde şehir var mı diye bakar.
- Varsa ID/documentId bilgisini döndürür.
- Yoksa yeni şehir oluşturur.
- Strapi v5 için documentId, v4/v5 uyumu için numeric id desteği de var.

Hocaya söyle:

Aynı şehir tekrar tekrar eklenmesin diye önce şehir var mı kontrol ediyorum. Varsa mevcut kaydı kullanıyorum, yoksa yeni kayıt açıyorum.

upload_image()

Ne yapar?

- Lokal görsel dosyasını açar.
- MIME type bilgisini otomatik belirler.
- /api/upload endpoint'ine multipart olarak gönderir.
- Strapi Media ID döndürür.

Önemli detay:

Upload yaparken Content-Type başlığını manuel vermiyorum. Çünkü multipart boundary bilgisini requests kütüphanesi kendisi doğru şekilde ayarlıyor.

create_place()

Ne yapar?

- Mekan daha önce eklenmiş mi kontrol eder.
- Varsa günceller.
- Yoksa yeni mekan oluşturur.
- Şehir ilişkisini city alanından bağlar.
- Görsel ID varsa cover_image alanına ekler.

Önemli ilişki payload'ı:

```
"city": {"connect": [city_relation_value]}
```

Hocaya söyle:

Mekanı şehre bağladığım kısım burası. Strapi relation alanı için connect yapısını kullanıyorum. Böylece mekan kaydı ilgili City kaydına bağlı geliyor.

9. Cloudinary ve Strapi Media Library

Dosya: strapi-backend/config/plugins.js

Amaç:

- Strapi upload provider'ını Cloudinary yapmak.
- Cloudinary bilgileri eksikse local upload'a düşmek.

Kontrol edilen env değişkenleri:

- UPLOAD_PROVIDER=cloudinary
- CLOUDINARY_NAME
- CLOUDINARY_KEY
- CLOUDINARY_SECRET

Hocaya söyle:

Strapi'de upload plugin'ini Cloudinary ile yapılandırdım. Python görseli /api/upload ile Strapi'ye gönderiyor. Strapi de upload provider olarak Cloudinary seçili olduğu için görseller Media Library üzerinden Cloudinary'de saklanıyor.

Kritik mantık:

```
provider: useCloudinary ? 'cloudinary' : 'local'
```

10. Render Deployment

Dosya: render.yaml

İki servis var:

1. ai-travel-guide-strapı

- Node servisi.
- Strapi backend'i çalıştırır.
- PostgreSQL veritabanına bağlanır.
- Cloudinary env değişkenlerini alır.

2. ai-travel-guide-frontend

- Python servisi.
- Streamlit uygulamasını çalıştırır.
- Strapi'ye token ve private hostport ile bağlanır.

Hocaya söyle:

Render'da backend ve frontend ayrı servisler. Backend Strapi ve PostgreSQL ile çalışıyor. Frontend Streamlit servisi ise Strapi'ye API üzerinden bağlanıyor.

Önemli Render değişkenleri:

- Backend:
- DATABASE_URL
- UPLOAD_PROVIDER
- CLOUDINARY_NAME
- CLOUDINARY_KEY
- CLOUDINARY_SECRET
- Frontend:
- STRAPI_API_TOKEN
- STRAPI_INTERNAL_HOSTPORT

11. frontend-streamlit/api.py Ne İş Yapar?

Bu dosya Streamlit'in Strapi'den veri çekmesini sağlar.

Önemli parçalar:

get_setting()

Önce environment variable okur, yoksa Streamlit secrets okur.

Hocaya söyle:

Uygulama hem lokal hem canlı ortamda çalışsın diye config değerlerini env veya Streamlit secrets üzerinden okuyorum.

StrapiClient.__init__()

Ne yapar?

- STRAPI_URL değerini hazırlar.
- Render private network için STRAPI_INTERNAL_HOSTPORT varsa onu kullanır.
- API token'ı temizler.
- Bearer header'ı hazırlar.

Token temizleme neden var?

Render'a token yanlışlıkla STRAPI_API_TOKEN=... veya Bearer ... şeklinde girilirse uygulama hata vermesin diye token normalize ediliyor.

fetch_cities()

Ne yapar?

- /api/cities endpoint'inden şehirleri çeker.
- Strapi v4 ve v5 JSON yapısını destekler.
- 403 hatasında kullanıcıya token/permission hatası olduğunu gösterir.

fetch_places(city_id)

Ne yapar?

- Seçilen şehre ait mekanları /api/places endpoint'inden çeker.
- populate=* ile görsel ilişkisini de getirir.
- Relation adı city veya eski yapıdan cities olabilir diye ikisini de dener.
- Görsel URL'i relative gelirse başına Strapi URL ekler.

Hocaya söyle:

Frontend tarafı Strapi'den gelen veriyi normalize ediyor. Böylece Strapi v4/v5 format farkı veya relation adındaki küçük farklar arayüzü bozmasın.

12. frontend-streamlit/app.py Ne İş Yapar?

Bu dosya kullanıcının gördüğü Streamlit arayüzüdür.

Ana özellikler:

- Sayfa başlığını ve görünümünü ayarlar.
- TR/EN dil seçimi yapar.
- Şehir seçme kutusu gösterir.
- Seçilen şehrin kısa bilgisini gösterir.
- O şehre ait mekanları kartlar halinde listeler.
- Görsel, puan, açıklama ve devamını oku alanı gösterir.

Önemli güvenlik fonksiyonu:

```
def safe_text(value) -> str:  
    return html.escape(str(value or ""))
```

Hocaya söyle:

Strapi'den gelen metinleri HTML içinde gösterdiğim için safe_text() ile escape ediyorum. Bu sayede metin içinde özel karakter veya HTML varsa sayfayı bozmaz.

Dil mantığı:

- TR seçilirse name, country, short_info, description_tr
- EN seçilirse name_en, country_en, short_info_en, description_en

Hocaya söyle:

Çok dilli gösterim için Strapi'de hem Türkçe hem İngilizce alanları tutuyorum. Streamlit'teki dil seçimine göre uygun alanı gösteriyorum.

13. Hocanın Sorabileceği Sorular ve Hazır Cevaplar

Soru: Veri nereden geliyor?

İlk veri automation/data/places.json dosyasından geliyor. Python scripti bu veriyi okuyor, yapay zeka ve çeviri ile zenginleştiriyor, sonra Strapi'ye API ile kaydediyor.

Soru: Yapay zeka nerede kullanıldı?

content_enricher.py içinde şehir ve mekan açıklamaları yapay zeka ile zenginleştiriliyor. image_generator.py içinde de mekanlara uygun turistik görseller oluşturuluyor.

Soru: Çeviri nerede yapılıyor?

translator.py içinde deep-translator kütüphanesiyle yapılıyor. translate_text() fonksiyonu source ve target dil kodu alıyor.

Soru: Görseller bilgisayarda mı kalıyor?

Hayır. Görsel önce upload edebilmek için lokal dosya olarak oluşturuluyor. Sonra strapi_api.py içindeki upload_image() fonksiyonu ile Strapi Media Library'ye gönderiliyor. Canlıda Cloudinary provider aktif olduğu için görseller Cloudinary üzerinde saklanıyor.

Soru: Strapi'ye güvenli bağlantı nasıl yapılıyor?

API isteklerinde Authorization: Bearer <token> header'ı kullanılıyor. Token .env veya Render environment variable üzerinden geliyor, kod içine yazılmıyor.

Soru: Şehir ve mekan ilişkisi nasıl kuruldu?

City.places one-to-many, Place.city many-to-one. Python'da mekan oluştururken "city": {"connect": [city_relation_value]} payload'ı ile mekan ilgili şehre bağlanıyor.

Soru: Aynı veri tekrar eklenirse duplicate olur mu?

Şehirlerde `get_or_create_city()` önce şehir var mı kontrol ediyor. Mekanlarda da `create_place()` önce aynı isimli mekan var mı kontrol ediyor, varsa update etmeye çalışıyor.

Soru: Frontend neden şehirleri bazen göremiyordu?

Canlı ortamda Strapi API public permission kapalıysa frontend token kullanmalı. Ayrıca Render'da frontend'in localhost yerine Strapi'nin public URL'i veya private `STRAPI_INTERNAL_HOSTPORT` adresini kullanması gerekiyor. Bu yüzden `api.py` içinde env kontrolü ve hata mesajları var.

Soru: `populate=*` neden kullanıldı?

Strapi ilişkili verileri varsayılan olarak her zaman detaylı döndürmez. `populate=*` ile `cover_image` gibi ilişkili medya alanlarını da response içine alıyorum.

Soru: Neden fallback kullandın?

Canlı projede API servisleri bazen hata verebilir. Bu yüzden metin üretmede Groq olmazsa Pollinations, görsel üretmede Pollinations olmazsa HuggingFace, o da olmazsa picsum fallback var. Amaç scriptin tek hatada tamamen durmaması.

14. En Kritik 10 Cümle

1. `main.py` projenin otomasyon akışını yöneten ana dosyadır.
2. Veriler `places.json` dosyasından okunur.
3. Metin zenginleştirme `content_enricher.py` içinde yapılır.
4. Çeviri `translator.py` içindeki `translate_text()` fonksiyonu ile yapılır.
5. Görsel üretimi `image_generator.py` içinde Pollinations/HuggingFace ile yapılır.
6. Strapi API bağlantısı `strapi_api.py` içindeki StrapiAPI sınıfıyla yapılır.
7. API güvenliği Authorization: Bearer TOKEN header'ı ile sağlanır.
8. Görseller `/api/upload endpoint'i` ile Strapi Media Library'ye yüklenir.
9. `Place.city` alanı ile mekanlar şehirlere bağlanır.
10. Streamlit arayüzü Strapi'den şehirleri ve mekanları çekip TR/EN olarak gösterir.

15. Çok Kısa Sunum Metni

Hocam bu projede üç ana katman var: Strapi backend, Python otomasyon ve Streamlit frontend. Python otomasyonu `places.json` dosyasındaki şehir ve mekan verilerini okuyor. Sonra açıklamaları yapay zeka ile zenginleştiriyor, `deep-translator` ile İngilizceye çeviriyor ve mekanlar için yapay zeka görselleri oluşturuyor. Oluşan görseller `strapi_api.py` üzerinden Strapi Media Library'ye yükleniyor. Mekan kayıtları da şehir ilişkisiyle birlikte Strapi'ye kaydediliyor. Strapi tarafında City ve Place koleksiyonları arasında one-to-many/many-to-one ilişki var. Frontend tarafında Streamlit, Strapi API'den şehirleri ve seçilen şehre ait mekanları çekiyor; kullanıcı TR veya EN dil seçerek içerikleri ve görselleri görebiliyor. Canlı ortamda Render kullanıldı, görsel saklama için de Cloudinary upload provider yapılandırıldı.

16. Takılırsan Kurtarıcı Cümleler

- "Bu kısmı modüler yazdım; çeviri, görsel üretimi ve Strapi API işlemleri ayrı dosyalarda."
- "Hata durumunda sistem tamamen çökmesin diye fallback mantığı kullandım."
- "Strapi v4 ve v5 response farklarını frontend tarafında normalize ettim."
- "Görseller sadece lokal kalmıyor, Media Library'ye upload ediliyor."
- "İlişkiyi Strapi schema tarafında kurdum, Python tarafında da connect ile bağlıyorum."
- "Canlı ortamda env değişkenleri Render üzerinden yönetiliyor, token kod içine yazılmadı."